

**LAPORAN PRAKTIKUM PEMROGRAMAN WEB MODUL 11:
PENGENALAN REACT**



**Universitas
Telkom**

Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

**UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS
TAHUN AJARAN 2026**

Tujuan Praktikum

Setelah mengikuti praktikum ini mahasiswa diharapkan dapat:

1. Memahami konsep dasar React sebagai library JavaScript untuk membangun antarmuka pengguna berbasis component dan declarative programming.
2. Memahami arsitektur UI berbasis component serta perbedaan pendekatan React dibandingkan manipulasi DOM tradisional.
3. Melakukan setup environment React dan membuat aplikasi React menggunakan Vite.
4. Membuat dan mengelola component menggunakan JSX serta memahami struktur file aplikasi React.
5. Menggunakan props untuk mengirim data antar component secara one-way data flow.
6. Mengelola state untuk membangun interaktivitas dasar pada aplikasi React.
7. Melakukan rendering list dan conditional rendering untuk menampilkan data secara dinamis.
8. Membangun aplikasi React sederhana yang menerapkan component, props, state, dan rendering dinamis secara terstruktur.

Alat dan Bahan

Untuk menjalankan modul ini, diperlukan komputer atau laptop dengan sistem operasi Windows, Linux, atau macOS serta browser modern seperti Google Chrome atau Mozilla Firefox.

Perangkat lunak yang digunakan meliputi:

- Node.js sebagai runtime JavaScript dan npm sebagai package manager.
- React Library untuk membangun antarmuka pengguna berbasis component.
- Vite sebagai build tool modern untuk membuat dan menjalankan proyek React.
- Code editor Visual Studio Code atau editor lain yang mendukung JavaScript dan React.
- Web browser untuk menjalankan serta melakukan debugging aplikasi React.

Bahan yang dibutuhkan:

- Project React daftar-aktivitas-mahasiswa.
- File JSX, CSS, dan konfigurasi npm.
- Materi Modul 11 Pengenalan React.

Langkah-Langkah Praktikum

Membuat folder project React bernama daftar-aktivitas-mahasiswa.

1. Menyiapkan file utama project, yaitu package.json, index.html, src/main.jsx, dan src/App.jsx.

2. Membuat component terpisah pada folder src/components, yaitu ActivityList.jsx dan ActivityItem.jsx.
3. Menyimpan data aktivitas awal dalam bentuk array pada state menggunakan useState.
4. Membuat input teks untuk menerima nama aktivitas baru dari pengguna.
5. Menambahkan aktivitas baru ke state menggunakan spread operator agar data lama tetap dipertahankan.
6. Menampilkan daftar aktivitas menggunakan rendering list dengan method map.
7. Menghapus aktivitas dari state menggunakan method filter saat tombol Hapus ditekan.
8. Menerapkan conditional rendering untuk menampilkan pesan Belum ada aktivitas saat daftar kosong.
9. Memberikan komentar pada kode program agar fungsi setiap bagian lebih mudah dipahami.
10. Menjalankan aplikasi dengan npm run dev dan menguji fitur tambah, hapus, serta kondisi list kosong.

Hasil dan Dokumentasi Program

Berikut seluruh kode program aplikasi Daftar Aktivitas Mahasiswa yang sudah diberi komentar penjelas:

File: package.json

```
{
  "name": "daftar-aktivitas-mahasiswa",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite --host 127.0.0.1",
    "build": "vite build",
    "preview": "vite preview --host 127.0.0.1"
  },
  "dependencies": {
    "@vitejs/plugin-react": "latest",
    "vite": "latest",
    "react": "latest",
    "react-dom": "latest"
  },
  "devDependencies": {}
}
```

File: index.html

```
<!doctype html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Daftar Aktivitas Mahasiswa</title>
  </head>
  <body>
    <!-- Elemen root menjadi tempat React merender seluruh tampilan aplikasi. -->
    <div id="root"></div>

    <!-- Script utama React yang memuat component App dan CSS. -->
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

File: src/main.jsx

```
import React from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './styles.css';

// Menghubungkan React ke elemen <div id="root"> yang ada di index.html.
createRoot(document.getElementById('root')).render(
  // StrictMode membantu menandai potensi masalah saat aplikasi dikembangkan.
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

File: src/App.jsx

```
import { useState } from 'react';
import ActivityList from './components/ActivityList.jsx';

// Data awal yang langsung tampil saat aplikasi pertama kali dibuka.
const initialActivities = [
  { id: 1, name: 'Mengikuti praktikum React' },
  { id: 2, name: 'Mengerjakan tugas pemrograman web' },
  { id: 3, name: 'Diskusi kelompok proyek akhir' }
];

// App berperan sebagai parent component yang menyimpan state utama aplikasi.
export default function App() {
  // State untuk menyimpan seluruh data aktivitas mahasiswa.
  const [activities, setActivities] = useState(initialActivities);

  // State untuk menyimpan teks yang sedang diketik pada input.
  const [activityName, setActivityName] = useState('');

  // Fungsi ini dijalankan saat form tambah aktivitas dikirim.
  function handleSubmit(event) {
    // Mencegah browser melakukan refresh halaman saat form submit.
    event.preventDefault();

    // trim() digunakan agar input berisi spasi saja tidak ikut ditambahkan.
    const trimmedName = activityName.trim();
    if (trimmedName === '') {
      return;
    }

    // Membuat object aktivitas baru dengan id unik dan nama dari input.
    const newActivity = {
      id: Date.now(),
      name: trimmedName
    };

    // Menambahkan aktivitas baru tanpa mengubah array state lama secara langsung.
    setActivities([...activities, newActivity]);

    // Mengosongkan kembali input setelah aktivitas berhasil ditambahkan.
    setActivityName('');
  }

  // Fungsi ini menghapus aktivitas berdasarkan id yang dikirim dari child component.
  function handleDeleteActivity(activityId) {
    // filter() menghasilkan array baru tanpa aktivitas yang id-nya dipilih.
    setActivities(activities.filter((activity) => activity.id !== activityId));
  }

  // Nilai boolean untuk menentukan apakah daftar aktivitas perlu ditampilkan.
  const hasActivities = activities.length > 0;

  return (
    <main className="app-shell">
      <div className="workspace">
        {/* Bagian samping hanya untuk identitas visual praktikum. */}
        <aside className="course-strip" aria-label="Info praktikum">
          <span>Pekan 11</span>
          <strong>React</strong>
          <small>state + props</small>
        </aside>

        <section className="activity-panel" aria-labelledby="app-title">
```

```

    { /* Elemen dekoratif agar panel terlihat seperti kertas binder. */ }
    <div className="paper-holes" aria-hidden="true">
      <span />
      <span />
      <span />
    </div>

    { /* Header menampilkan judul aplikasi dan jumlah aktivitas saat ini. */ }
    <div className="header">
      <p className="eyebrow">Catatan Praktikum</p>
      <h1 id="app-title">Daftar Aktivitas Mahasiswa</h1>
      <p className="summary">
        <span>{activities.length}</span> aktivitas tercatat
      </p>
    </div>

    { /* Form untuk menerima input aktivitas baru dari pengguna. */ }
    <form className="activity-form" onSubmit={handleSubmit}>
      <label htmlFor="activityName">Nama aktivitas</label>
      <div className="input-row">
        <input
          id="activityName"
          type="text"
          // value membuat input dikontrol oleh state React.
          value={activityName}
          // Setiap perubahan input langsung disimpan ke state activityName.
          onChange={(event) => setActivityName(event.target.value)}
          placeholder="Contoh: Membaca materi React"
        />
        <button type="submit">
          <span aria-hidden="true">+</span>
          Tambah
        </button>
      </div>
    </form>

    { /* Conditional rendering: tampilkan list jika ada data, pesan kosong jika tidak
ada. */ }
    {hasActivities ? (
      <ActivityList
        // Data aktivitas dikirim ke child component melalui props.
        activities={activities}
        // Fungsi hapus juga dikirim sebagai props agar item bisa memicu perubahan state
        onDeleteActivity={handleDeleteActivity}
      />
    ) : (
      <p className="empty-message">Belum ada aktivitas</p>
    )}
  </section>
</div>
</main>
);
}

```

File: src/components/ActivityList.jsx

```

import ActivityItem from './ActivityItem.jsx';

// ActivityList menerima data dari App dan menampilkannya sebagai daftar.
export default function ActivityList({ activities, onDeleteActivity }) {
  return (
    <ul className="activity-list">
      { /* map() digunakan untuk mengubah setiap data aktivitas menjadi component ActivityItem. */ }
      {activities.map((activity, index) => (
        <ActivityItem
          // key membantu React mengenali setiap item saat daftar berubah.
          key={activity.id}
          // activity berisi data satu aktivitas yang akan ditampilkan.
          activity={activity}
          // index digunakan untuk menampilkan nomor urut aktivitas.
          index={index}
          // Fungsi hapus diteruskan lagi ke ActivityItem.
          onDeleteActivity={onDeleteActivity}
        />
      )}
    </ul>
  );
}

```

```

    ))}
  </ul>
);
}

```

File: src/components/ActivityItem.jsx

```

// ActivityItem bertugas menampilkan satu aktivitas beserta tombol hapusnya.
export default function ActivityItem({ activity, index, onDeleteActivity }) {
  return (
    <li className="activity-item">
      {/* Menampilkan nomor urut dengan format dua digit, misalnya 01, 02, 03. */}
      <span className="activity-number">{String(index + 1).padStart(2, '0')}</span>

      {/* Menampilkan nama aktivitas dari props activity. */}
      <span className="activity-name">{activity.name}</span>

      <button
        type="button"
        className="delete-button"
        // aria-label membuat tombol lebih jelas untuk pembaca layar.
        aria-label={`Hapus aktivitas ${activity.name}`}
        // Saat diklik, component mengirim id aktivitas ke parent untuk dihapus dari state.
        onClick={() => onDeleteActivity(activity.id)}
      >
        <span aria-hidden="true">x</span>
        Hapus
      </button>
    </li>
  );
}

```

File: src/styles.css

```

/* Pengaturan dasar warna, font, dan rendering teks untuk seluruh aplikasi. */
:root {
  font-family:
    "Segoe UI", "Trebuchet MS", ui-sans-serif, system-ui, -apple-system,
    BlinkMacSystemFont, sans-serif;
  color: #192027;
  background: #e7e0d2;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

/* Membuat perhitungan ukuran elemen lebih konsisten. */
* {
  box-sizing: border-box;
}

/* Menghapus margin bawaan browser dan menjaga lebar minimum halaman. */
body {
  margin: 0;
  min-width: 320px;
}

/* Memastikan input dan button memakai font yang sama dengan halaman. */
button,
input {
  font: inherit;
}

button {
  border: 0;
}

/* Layout utama untuk menempatkan aplikasi tepat di tengah layar. */
.app-shell {
  min-height: 100vh;
  display: grid;
  place-items: center;
  padding: 36px 18px;
  background:
    linear-gradient(90deg, rgba(37, 56, 67, 0.05) 1px, transparent 1px),
    linear-gradient(0deg, rgba(37, 56, 67, 0.05) 1px, transparent 1px),

```

```

    #e7e0d2;
    background-size: 28px 28px;
}

/* Container besar yang membagi tampilan menjadi strip kiri dan panel utama. */
.workspace {
    position: relative;
    display: grid;
    grid-template-columns: 112px minmax(0, 780px);
    width: min(100%, 920px);
    filter: drop-shadow(0 24px 28px rgba(61, 48, 35, 0.18));
}

/* Strip kiri berisi identitas praktikum dan memberi karakter visual pada UI. */
.course-strip {
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    min-height: 600px;
    padding: 28px 18px;
    color: #f8f0dd;
    background: #243b39;
    border: 2px solid #172925;
    border-right: 0;
    border-radius: 10px 0 0 10px;
    box-shadow: inset -10px 0 0 rgba(255, 255, 255, 0.06);
    text-transform: uppercase;
}

/* Teks pada strip dibuat vertikal agar terlihat seperti label binder. */
.course-strip span,
.course-strip small {
    font-size: 0.78rem;
    font-weight: 800;
    letter-spacing: 0.08em;
    writing-mode: vertical-rl;
    transform: rotate(180deg);
}

/* Badge React di tengah strip kiri. */
.course-strip strong {
    align-self: center;
    width: 68px;
    height: 68px;
    display: grid;
    place-items: center;
    color: #243b39;
    background: #f5c64b;
    border: 2px solid #172925;
    border-radius: 50%;
    font-size: 0.95rem;
}

/* Panel utama dibuat seperti lembar catatan praktikum. */
.activity-panel {
    position: relative;
    min-height: 600px;
    overflow: hidden;
    padding: 44px 46px 42px 62px;
    border: 2px solid #25333d;
    border-radius: 0 10px 10px 0;
    background:
        linear-gradient(90deg, transparent 0 58px, #d8665b 58px 60px, transparent 60px),
        repeating-linear-gradient(#fffaf0 0 34px, #d8e6ee 35px 36px);
}

/* Elemen dekoratif seperti sticky note kecil di pojok bawah. */
.activity-panel::after {
    content: "";
    position: absolute;
    right: 24px;
    bottom: 24px;
    width: 112px;
    height: 46px;
    background: rgba(245, 198, 75, 0.42);
    border: 1px solid rgba(151, 106, 26, 0.22);
}

```

```

    transform: rotate(-3deg);
    pointer-events: none;
}

/* Lubang kertas binder sebagai dekorasi visual. */
.paper-holes {
    position: absolute;
    left: 20px;
    top: 74px;
    display: grid;
    gap: 120px;
}

/* Bentuk tiap lubang pada panel kertas. */
.paper-holes span {
    width: 18px;
    height: 18px;
    display: block;
    border: 2px solid #bacbd1;
    border-radius: 50%;
    background: #e7e0d2;
    box-shadow: inset 0 2px 5px rgba(37, 51, 61, 0.22);
}

/* Header berisi label, judul aplikasi, dan total aktivitas. */
.header {
    position: relative;
    display: grid;
    grid-template-columns: 1fr auto;
    gap: 8px 24px;
    align-items: end;
    padding-bottom: 24px;
}

/* Label kecil di atas judul. */
.eyebrow {
    grid-column: 1 / -1;
    width: fit-content;
    margin: 0;
    padding: 5px 12px;
    color: #1e302e;
    background: #b9d5c8;
    border: 1px solid #608274;
    border-radius: 999px;
    font-size: 0.76rem;
    font-weight: 800;
    text-transform: uppercase;
}

/* Judul utama aplikasi. */
h1 {
    max-width: 520px;
    margin: 0;
    color: #182a33;
    font-size: clamp(2rem, 4.5vw, 3.45rem);
    line-height: 0.98;
    letter-spacing: 0;
}

/* Kotak ringkasan jumlah aktivitas. */
.summary {
    justify-self: end;
    min-width: 136px;
    margin: 0;
    padding: 12px 14px;
    color: #20323b;
    background: #f7d95e;
    border: 2px solid #25333d;
    border-radius: 6px;
    text-align: center;
    transform: rotate(2deg);
}

/* Angka total aktivitas dibuat lebih besar agar mudah dibaca. */
.summary span {
    display: block;

```



```

    font-size: 1.8rem;
    font-weight: 900;
    line-height: 1;
}

/* Form input aktivitas baru. */
.activity-form {
    position: relative;
    z-index: 1;
    display: grid;
    gap: 10px;
    margin-top: 14px;
}

/* Label input dibuat tebal sebagai penanda field. */
.activity-form label {
    width: fit-content;
    color: #34444b;
    font-size: 0.92rem;
    font-weight: 800;
}

/* Baris input dan tombol tambah dibuat sejajar di layar lebar. */
.input-row {
    display: grid;
    grid-template-columns: 1fr auto;
    gap: 12px;
}

/* Field input untuk mengetik nama aktivitas. */
.input-row input {
    width: 100%;
    min-height: 52px;
    border: 2px solid #25333d;
    border-radius: 7px;
    padding: 0 16px;
    color: #192027;
    background: rgba(255, 255, 255, 0.72);
    box-shadow: 4px 4px 0 #25333d;
}

/* Efek fokus agar pengguna tahu input sedang aktif. */
.input-row input:focus {
    outline: 3px solid rgba(91, 130, 170, 0.28);
}

/* Tombol tambah aktivitas. */
.input-row button {
    min-height: 52px;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    padding: 0 20px;
    color: #fdf8e8;
    background: #315c6c;
    border: 2px solid #25333d;
    border-radius: 7px;
    box-shadow: 4px 4px 0 #25333d;
    cursor: pointer;
    font-weight: 900;
}

/* Ikon plus kecil di dalam tombol tambah. */
.input-row button span {
    width: 22px;
    height: 22px;
    display: grid;
    place-items: center;
    color: #315c6c;
    background: #fdf8e8;
    border-radius: 50%;
    line-height: 1;
}

```

```

/* Efek hover sederhana agar tombol terasa interaktif. */
.input-row button:hover {
  transform: translate(1px, 1px);
  box-shadow: 3px 3px 0 #25333d;
}

/* Container daftar aktivitas. */
.activity-list {
  position: relative;
  z-index: 1;
  display: grid;
  gap: 13px;
  list-style: none;
  margin: 30px 0 0;
  padding: 0;
}

/* Satu baris item aktivitas. */
.activity-item {
  display: grid;
  grid-template-columns: 56px 1fr auto;
  align-items: center;
  gap: 14px;
  min-height: 62px;
  padding: 11px 12px;
  color: #1a252b;
  background: #fffdf7;
  border: 2px solid #25333d;
  border-radius: 7px;
  box-shadow: 6px 6px 0 rgba(49, 92, 108, 0.22);
}

/* Variasi warna dan rotasi ringan agar daftar tidak terlihat kaku. */
.activity-item:nth-child(2n) {
  background: #f1f7f4;
  transform: rotate(-0.4deg);
}

/* Variasi rotasi untuk item bernomor ganjil. */
.activity-item:nth-child(2n + 1) {
  transform: rotate(0.35deg);
}

/* Nomor urut aktivitas. */
.activity-number {
  height: 38px;
  display: grid;
  place-items: center;
  color: #25333d;
  background: #f5c64b;
  border: 2px solid #25333d;
  border-radius: 999px;
  font-weight: 900;
}

/* Nama aktivitas dibuat aman saat teks terlalu panjang. */
.activity-name {
  overflow-wrap: anywhere;
  font-weight: 750;
}

/* Tombol untuk menghapus satu aktivitas. */
.delete-button {
  min-height: 38px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  gap: 7px;
  padding: 0 13px;
  color: #702f2c;
  background: #ffd9d1;
  border: 2px solid #8c4944;
  border-radius: 999px;
  cursor: pointer;
  font-weight: 900;
}

```

```

/* Simbol x pada tombol hapus. */
.delete-button span {
  font-weight: 900;
  line-height: 1;
}

/* Efek hover pada tombol hapus. */
.delete-button:hover {
  color: #fff7ef;
  background: #9c3d38;
}

/* Pesan yang tampil ketika array aktivitas kosong. */
.empty-message {
  position: relative;
  z-index: 1;
  margin: 32px auto 0;
  width: min(100%, 390px);
  border: 2px dashed #25333d;
  border-radius: 7px;
  padding: 24px;
  color: #536168;
  text-align: center;
  background: rgba(255, 255, 255, 0.6);
  font-weight: 800;
}

/* Responsif untuk tablet: strip kiri dipindahkan ke atas. */
@media (max-width: 760px) {
  .workspace {
    grid-template-columns: 1fr;
  }

  .course-strip {
    min-height: 0;
    flex-direction: row;
    align-items: center;
    gap: 16px;
    padding: 14px 18px;
    border-right: 2px solid #172925;
    border-bottom: 0;
    border-radius: 10px 10px 0 0;
  }

  .course-strip span,
  .course-strip small {
    writing-mode: initial;
    transform: none;
  }

  .course-strip strong {
    width: auto;
    height: auto;
    padding: 8px 14px;
    border-radius: 999px;
  }

  .activity-panel {
    min-height: 560px;
    padding: 34px 22px 30px 34px;
    border-radius: 0 0 10px 10px;
  }

  .header {
    grid-template-columns: 1fr;
  }

  .summary {
    justify-self: start;
    transform: none;
  }

  .paper-holes {
    display: none;
  }
}

```

```

}
}

/* Responsif untuk layar kecil: input, item, dan tombol disusun vertikal. */
@media (max-width: 560px) {
  .app-shell {
    padding: 18px 10px;
  }

  .activity-panel {
    background:
      linear-gradient(90deg, transparent 0 24px, #d8665b 24px 26px, transparent 26px),
      repeating-linear-gradient(#ffffaf 0 34px, #d8e6ee 35px 36px);
  }

  .input-row,
  .activity-item {
    grid-template-columns: 1fr;
  }

  .activity-number {
    width: 48px;
  }

  .delete-button,
  .input-row button {
    width: 100%;
  }
}
}

```

Output



Gambar 1. Tampilan awal aplikasi daftar aktivitas mahasiswa



Gambar 2. Tampilan setelah aktivitas baru ditambahkan



Gambar 3. Tampilan conditional rendering saat daftar aktivitas kosong

Analisis dan Pembahasan

Aplikasi ini menggunakan React component sehingga struktur tampilan dibagi menjadi beberapa bagian, yaitu App sebagai parent component serta ActivityList dan ActivityItem sebagai child component. Data aktivitas disimpan pada state ``activities``, sehingga setiap perubahan data akan otomatis memperbarui tampilan melalui mekanisme re-render pada React. Selain itu, input aktivitas memanfaatkan state ``activityName`` untuk menyimpan nilai yang diketik pengguna sebelum data ditambahkan ke dalam daftar aktivitas. Proses rendering daftar aktivitas dilakukan menggunakan method ``map`` pada component ``ActivityList``, sehingga setiap data aktivitas dapat ditampilkan sebagai item tersendiri secara dinamis.

Penghapusan aktivitas dilakukan menggunakan method ``filter``, sehingga state baru tidak lagi memuat item dengan ``id`` yang dipilih. Aplikasi juga menerapkan conditional rendering untuk menampilkan daftar aktivitas ketika data tersedia, serta menampilkan pesan “Belum ada aktivitas” apabila array state kosong. Dalam implementasinya, props digunakan untuk mengirim data dan fungsi dari component App ke ActivityList dan ActivityItem sesuai dengan konsep one-way data flow pada React. Selain itu, komentar pada kode ditambahkan untuk membantu menjelaskan fungsi state, props, event handler, rendering list, serta styling sehingga program menjadi lebih mudah dipahami dan dipelajari.

Kesimpulan

Praktikum berhasil diimplementasikan. Aplikasi React sederhana dapat menampilkan daftar aktivitas, menambahkan aktivitas baru, menghapus aktivitas, serta menampilkan pesan khusus saat daftar kosong. Konsep component, props, state, rendering list, conditional rendering, dan dokumentasi kode melalui komentar berhasil diterapkan pada aplikasi Daftar Aktivitas Mahasiswa.

Lampiran

<https://github.com/AriqAhmadNayaka/WebPro4904/tree/AndhikaYuridho-707012500059>